

Architectures Parallèles

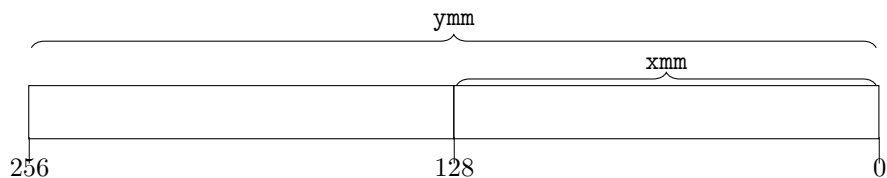
TP4: x86_64 assembly - SIMD with SSE and AVX (part 2)

UVSQ - M1 CHPS

0 Cheatsheet

0.1 Registers

We have seen how SSE first introduced SIMD compute capabilities on x86 machines, through the 128 bits wide XMM registers. Those registers were later extended by AVX to 256 bits and named YMM:



0.2 Instructions

Here are more AVX SIMD instructions:

Instruction	Description	C equivalent
<code>leaq r, m</code>	Compute address of memory operand <code>m</code> and copy it to <code>r</code>	<code>r = m;</code>
<code>vaddpd x1, x2, x3</code>	Double-precision packed FP add (AVX, 128 bits)	<code>x31=x11+x21;</code> <code>x12=x22+ x32;</code>
<code>vaddpd y1, y2, y3</code>	Double-precision packed FP add (AVX, 256 bits)	<code>y11=y21+y31; ...;</code> <code>y14=y24+y34;</code>
<code>vaddps x1, x2, x3</code>	Single-precision packed FP add (AVX, 128 bits)	<code>x11=x21+x31; ...;</code> <code>x14=x24+x34;</code>
<code>vaddps y1, y2, y3</code>	Single-precision packed FP add (AVX, 256 bits)	<code>y11=y21+y31; ...;</code> <code>y18=y28+y38;</code>
<code>movdqu x1, m</code>	Load packed integers into an XMM register	<code>x11=m; ...;</code>
<code>movdqu y1, m</code>	Load packed integers into an YMM register	<code>y11=m; ...;</code>
<code>vpbroadcastq y, m</code>	Broadcast 64 bit integer stored at <code>m</code> into <code>y</code>	<code>x11=m; ...;</code>
<code>vgatherdpd y1, m, x2</code>	Conditionally gather FP64 elements from address <code>m</code> into <code>y1</code> using mask <code>x2</code> and doubleword indices	See below
<code>vgatherqpd y1, m, y2</code>	Conditionally gather FP64 elements from address <code>m</code> into <code>y1</code> using mask <code>x2</code> and quadword indices	See below
<code>vgatherqpd x1, m, x2</code>	Conditionally gather FP64 elements from address <code>m</code> into <code>x1</code> using mask <code>x2</code> and quadword indices	See below
<code>vgatherqps x1, m, x2</code>	Conditionally gather FP32 elements from address <code>m</code> into <code>x1</code> using mask <code>x2</code> and quadword indices	See below

Scatter operations perform conditional loads into elements of a vector register from a base address plus some offset indices. The memory operand has to be of the form `[a+(x/y)*k]`, where addresses corresponding to the indices given by each elements of the `x` or `y` register will be loaded into the destination register.

Regarding the mask operand (the last register), it controls whether each element should be loaded or not. To enable an element of the mask, its most significant bit should be set to 1. In this lab, we will always enable all elements. One easy way to do so is by setting a mask element to all 1s. In C, this can be done with `~(maskelem & 0)`. This element can then be broadcasted to all elements of the mask vector:

11111111...1	11111111...1	11111111...1	11111111...1
--------------	--------------	--------------	--------------

For instance, doing a `"vgatherqpd ymm0, [m+ymm1*8], ymm2"` with the ymm2 mask register set as above, the following array of 64 bits elements at base address `m`

m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7	m_8	m_9
-------	-------	-------	-------	-------	-------	-------	-------	-------	-------

0xm

and the following ymm1 indices vector to index `m`

4	7	3	1
---	---	---	---

256 192 128 64 0

would result into the following ymm0 vector

m_4	m_7	m_3	m_1
-------	-------	-------	-------

256 192 128 64 0

Note: The "opposite" operation of gather loads are **scatter** stores. It stores elements of a vector register to a base address plus some offsets indices. However, scatters have only been introduced to x86 with the AVX-512 instruction set extension, which is not available on all modern machines. To see which AVX extensions are supported by your machine:

```
$ lscpu | grep --color avx
```

0.3 2D array storage

Consider the example matrix

$$A = \begin{pmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \end{pmatrix}$$

It could be stored in memory as a two dimensional array `A[m][n]` ($m = 4$ and $n = 3$) with two different conventions:

- **row major**, by storing elements of the same **rows** contiguously. It is the storage mode used by C and C++.

a_{11}	a_{12}	a_{13}	a_{14}	a_{21}	a_{22}	a_{23}	a_{24}	a_{31}	a_{32}	a_{33}	a_{34}
----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------

The element `A[i][j]` has address `A + i*n + j`.

- **column major**, by storing elements of the same **columns** contiguously. It is the storage mode used by Fortran.

a_{11}	a_{21}	a_{31}	a_{12}	a_{22}	a_{32}	a_{13}	a_{23}	a_{33}	a_{14}	a_{24}	a_{34}
----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------

The element `A[i][j]` has address `A + j*m + i`.

Note that those two storage modes can also be generalized to n -dimensional arrays.

1 Simple gather loads

Consider the following C loop:

```
1 void gather_fixedsize(double a[4], size_t id[4], double b[4]) {
2     for(size_t i=0; i<4; ++i) {
3         a[i] = b[id[i]];
4     }
5 }
```

1.1 Fixed size

Using a single `vgatherqpd` instruction, write an equivalent code in ASM or inline ASM.

Consider the following C loop:

```
1 void gather(size_t n, double a[n], size_t id[n], double b[n]) {
2     for(size_t i=0; i<n; ++i) {
3         a[i] = b[id[i]];
4     }
5 }
```

1.2 Size n , scalar

Start by writing a scalar version of this loop in ASM or inline ASM.

1.3 Size n , vectorized

Write a vectorized version of this loop using gather loads.

2 Matrix transpose

Consider the following C loop computing a matrix transpose:

```
1 void transpose(uint64_t m, uint64_t n, double a[m][n], double b[n][m]) {
2     for(size_t i=0; i<m; ++i) {
3         for(size_t j=1; j<n; ++j) {
4             b[i][j] = a[j][i];
5         }
6     }
7 }
```

2.1

Start by writing a scalar version of this loop nest in ASM or inline ASM.

2.2

Using gather loads, write a vectorized version of this loop.

Note how this could also be written using scatter stores. However, because AVX-512 support is not widely spread, we will use the version with gather loads.

Tip: Because the gather indices are going to be the same for all loop iterations, you are allowed to compute them outside your ASM code and load them at the very beginning.